Assignment 2
COMP3100/6105: Distributed Systems, S1, 2026
SID:47943645 and 47298774 Name: Erick Ruwona and Yashaswi Shilpakar

1 Introduction

Distributed systems often execute large numbers of independent jobs on shared pools of heterogeneous servers [2, 5]. A scheduler decides where each job should run. This decision has a direct effect on average turnaround time, resource allocation/utilization, and rental cost. In an environment that emulates a data-center, assigning every job to the first server that can run it is a simple method that comes with the downside of potentially creating queues on unsuitable machines. Conversely, there is the downside of activating too many servers, which could possibly reduce waiting time while increasing rental cost. A good ds-sim client should be able to balance short job completion times with reasonable resource use. This is especially important because ds-sim configurations contain different server types, job sizes and workload patterns. A scheduling method that performs well in one configuration may not perform as well in another if it relies too heavily on one fixed rule. Hence, the client needs to make decisions dynamically based on the current availability of servers and the estimated load already assigned to them. The AEF-CF approach was designed with this in mind, using both live server queries and local waiting-time tracking to guide each scheduling decision. This allows the algorithm to respond to changing system conditions while still remaining simple and efficient to implement.

This report presents a client-side scheduling algorithm for ds-sim/ds-server [1]. The goal of the assignment is to design and implement a client that schedules jobs to servers with lower average turnaround time than the pre-existing baseline client algorithms, while not sacrificing resource utilization and rental cost excessively. The implemented algorithm is called *Available Earliest-Finish with Capable Fallback* (AEF-CF). It first attempts to use currently available servers, then falls back to capable servers, and uses a local wait-time estimate to select the server expected to finish the job earliest. The report follows this structure: Section 2 defines the scheduling problem and the objective function. Section 3 describes the algorithm and how it functions alongside a small example. Section 4 Summarises how we went about the implementation and data structures 5 evaluates the algorithm against ATL, FF, BF, FC and FAFC. Section 6 concludes the report and suggests improvements.

2 Problem definition

The scheduling challenge is an online job-to-server assignment problem [3, 4]. Jobs arrive over time as `JOBN` messages. Each job j has a submit time r_j, an estimated runtime p_j, and resource requirements for cores, memory and disk. Each server has a type, identifier, state, boot time, capacity and current queue status.

When a job arrives, the client must select a server and issue a `SCHD` command. The client cannot change the schedule later, so each decision must be based on the current response from ds-server and any local state stored by the client.

The main objective is to minimise average turnaround time:

$$\min\ \overline{T} = \frac{1}{n}\sum_{j=1}^{n}(C_j - r_j),$$

where C_j is the completion time of job j, r_j is the job submit time, and n is the number of completed jobs.

This objective is subject to feasibility constraints: the selected server must have at least the cores, memory and disk required by the job. Some secondary metrics are also considered, including resource utilisation and total rental cost.

These objectives conflict because always using many idle servers can reduce queueing delay, but may raise rental cost. On the other hand, packing jobs onto fewer servers may lower cost, but can increase waiting time. The implemented algorithm therefore prioritises turnaround time, then uses smaller-server tie-breaking to avoid unnecessary over-provisioning.

3 Algorithm description

3.1 Available Earliest-Finish with Capable Fallback

The AEF-CF algorithm is a greedy online heuristic. It estimates the completion time of placing a new job on a server using the client-side field `waitTime`. Since the current job's estimated runtime is the same for all candidate servers, selecting the lowest `waitTime` corresponds to selecting the lowest expected finish time. When two servers have the same predicted wait time, the algorithm chooses the server with the fewest cores. This tie-breaker acts as a lightweight best-fit rule that minimises the possibility of wasting larger machines on smaller jobs.

For each incoming job, the algorithm follows the steps below:

1. Query `GETS Avail cores memory disk`. If at least one available server is returned, choose the server with the shortest `waitTime`. If there is a tie, the server with the smaller core count is selected.
2. If no matching available server is found, query `GETS Capable cores memory disk`. The algorithm then chooses the capable server with the shortest `waitTime`, again breaking ties using the lower core count.
3. If the aforementioned queries return no valid candidates, the algorithm chooses a locally cached server type that can fit the job and has the shortest projected wait time.
4. As a last resort, the algorithm uses the largest known server. This is only intended to prevent failure when the server list or candidate query is unexpectedly empty.
5. After a successful scheduling decision the job's estimated runtime is added to the selected server's `waitTime`. Upon job completion, the completed job's runtime is subtracted from that server's local estimate.

The approach is to prefer a server that can start now, while avoiding larger idle servers if a smaller idle server is equally suitable. If no server is currently available, the job is placed where the local queue estimate is shortest.

3.2 Example scheduling scenario

Table 1 shows a small configuration used to visualise the algorithm. All servers are initially idle and have `waitTime = 0`. The small servers are preferred for small jobs because they have the same estimated waiting time as larger servers but fewer cores.

Server type	Count	Cores	Memory	Disk
small	2	2	4096	8000
medium	1	4	8192	16000
large	1	8	16384	32000

Table 1: Sample server configuration for the scheduling example.

Job	Requirements	Runtime	Candidate situation	Chosen server	Reason
J1	2 cores	20	all idle	small-0	lowest wait; smallest fit
J2	4 cores	30	medium/large idle	medium-0	smallest server that fits
J3	2 cores	10	small-1 idle	small-1	immediate start
J4	2 cores	15	no idle small; capable queried	small-1	shortest estimated wait

Table 2: Example schedule produced by AEF-CF.

After J1 is scheduled to `small-0`, that server's estimated wait becomes 20. J2 cannot fit on a small server, so `medium-0` is selected and its estimated wait becomes 30. J3 uses `small-1`. When J4 arrives, if all currently suitable available servers are busy, the capable-server fallback compares the local estimates and selects `small-1`, which has an estimated wait of 10 instead of `small-0`'s 20 or `medium-0`'s 30. This reduces expected queueing delay without immediately using the larger server.

4 Implementation details

The client is implemented in Java using a TCP socket to `localhost:50000`, following the client-server structure used by ds-sim [1]. It follows the ds-server protocol sequence `HELO`, `AUTH`, `GETS All`, repeated `REDY/SCHD` messages, and finally `QUIT`. Server information is stored in `HashMap<String, ArrayList<Server>> allServers`

and each groups cached servers by type. Each `Server` object stores the server type, id, state, boot time, capacity, waiting job count, running job count and a local `waitTime`. Jobs are represented by a `Job` object containing submit time, job id, resource requirements and estimated runtime.

The main selection functions are `getAvailableServers`, `getCapableServers`, `selectEarliestFinish`, `bestServer` and `largestServer`. `selectEarliestFinish` scans the candidate list linearly and keeps the server with the minimum estimated wait, using smaller core count as a tie-break. The per-job decision cost is therefore $O(a + c + s)$ in the worst case, where a is the number of available servers returned, c is the number of capable servers returned, and s is the number of cached servers checked in the fallback. In practice, network communication with ds-server dominates the runtime, while the local computations are small.

The implementation deliberately avoids complex global optimisation because jobs arrive online and must be scheduled immediately. Its main local state is the estimated wait time, which approximates the amount of queued work on each server. This estimate is updated after scheduling and after job completion. The benefit is low overhead; the limitation is that the estimate can become inaccurate if runtime estimates are poor or if completion tracking is inconsistent.

5 Evaluation

5.1 Simulation setup

The algorithm was evaluated using 20 ds-sim configuration files covering different numbers of servers and workload patterns, including `long`, `med`, `short`, `high` and `alt` cases. The simulations were run locally using ds-server on `localhost` with port `50000`. The Java client was tested through the supplied testing script using the command:

```
python3 ./ds_test.py "java -cp ./Client Client" -n -p 50000 -c TestConfigs
```

after compilation of the java client.
This command starts the ds-sim testing process, connects the client to the local ds-server instance, and runs the client against the configurations stored in the `TestConfigs` directory. Results were compared against the baseline algorithms ATL, FF, BF, FC and FAFC. The three reported metrics were average turnaround time, resource utilisation and total rental cost. Lower turnaround time and lower rental cost are better, while higher utilisation is generally better provided it does not create excessive queueing delay.

5.2 Results and discussion

Algorithm	Avg. turnaround	Normalised vs FAFC	Avg. utilisation (%)	Avg. rental cost	Summary
ATL	294261.95	201.1566	100.00	391.03	very high utilisation, poor turnaround
FF	1830.30	1.2512	71.53	551.55	simple and fast, slower than AEF-CF
BF	2247.70	1.5365	67.54	550.03	good packing, but worse turnaround
FC	328410.70	224.5006	97.67	378.15	low cost, poor responsiveness
FAFC	1462.85	1.0000	71.86	551.35	best average turnaround baseline
AEF-CF	1491.50	1.0196	68.05	549.95	close to FAFC, cheaper but lower utilisation

Table 3: Average performance across the 20 supplied configurations.

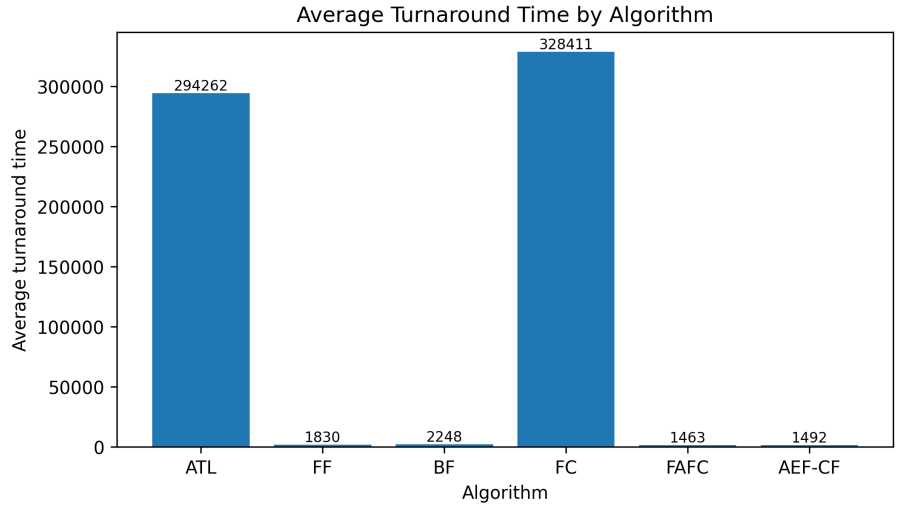

Figure 1: Average turnaround time.

Table 3 shows that AEF-CF achieved an average turnaround time of 1491.50. This was much lower than ATL, FC, FF and BF. Compared with FF, AEF-CF reduced average turnaround time by about 18.5%; compared with BF, it reduced turnaround time by about 33.6%. Compared with FAFC, AEF-CF was slightly worse in turnaround time: 1491.50 versus 1462.85, a gap of about 2.0%. This indicates that the local wait-time heuristic is competitive, but not quite as strong as FAFC on the tested workloads.

The algorithm's resource utilisation averaged 68.05%. This is lower than FAFC's 71.86% and FF's 71.53%, but slightly higher than BF's 67.54%. The lower utilisation suggests that AEF-CF sometimes spreads work or selects smaller servers in a way that does not keep resources as consistently busy as FAFC. However, its total rental cost averaged 549.95, which is slightly cheaper than FAFC's 551.35 and FF's 551.55. The cost difference is small, but it shows that the smaller-core tie-break did not increase rental cost relative to the strongest turnaround baseline.

The main advantage of AEF-CF is that it is simple, fast and general. It does not assume any particular configuration file and can handle arbitrary server types as long as ds-server returns them through `GETS All`, `GETS Avail` and `GETS Capable`. It also avoids overloading the first capable server by considering local queue estimates. The main disadvantage is that it is greedy and uses only an approximate wait model. It does not explicitly model booting cost, detailed remaining runtime of queued jobs, or future job arrivals. These limitations explain why FAFC remains slightly ahead on average turnaround time.

6 Conclusion

This report presented AEF-CF, a greedy ds-sim scheduling algorithm that prioritises low estimated waiting time while using smaller-server tie-breaking to control resource waste. The algorithm substantially outperformed ATL, FC, FF and BF in average turnaround time and came within about 2% of FAFC. It also achieved a slightly lower average rental cost than FAFC, although with lower utilisation. The results suggest that local wait-time tracking is an effective improvement over simple first-fit and best-fit policies. Future improvements should combine this heuristic with more accurate remaining-time estimates, explicit boot-state handling and adaptive tie-breaks that respond to workload length and server utilisation.

7 References

References

[1] distsys-MQ, *ds-sim: Distributed Systems Simulator*, GitHub repository. Accessed: 30 May 2026. [Online]. Available: `https://github.com/distsys-MQ/ds-sim`

[2] B. Hindman, A. Konwinski, M. Zaharia, A. Ghodsi, A. D. Joseph, R. Katz, S. Shenker, and I. Stoica, "Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center," in *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 11)*, Boston, MA, USA, 2011.

[3] R. L. Graham, “Bounds for Certain Multiprocessing Anomalies,” *Bell System Technical Journal*, vol. 45, no. 9, pp. 1563–1581, 1966.

[4] L. Epstein, “List Scheduling,” in *Encyclopedia of Algorithms*, M.-Y. Kao, Ed. Boston, MA, USA: Springer, 2008, pp. 455–457.

[5] K. Ousterhout, P. Wendell, M. Zaharia, and I. Stoica, “Sparrow: Distributed, Low Latency Scheduling,” in *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, 2013.